Quantum Min/Max Searching

Lecture 3

Mingyu Lee
Seoul National University

red1108@snu.ac.kr
+82 010-2857-3320 (only text) / +01 530-760-8690

2026.04.24(FRI) 16:30 KST

Welcome back

Week 3. Thanks for coming back!

Last two weeks we built up the query model and Grover.
This week we take Grover to its first real application:

Finding the minimum of an unsorted list.

All slides remain on GitHub: `https://github.com/red1108/lectures`

Today's Plan

- From "is there a marked item?" to "what is the best item?"
- Quantum exponential searching (BBHT): Grover when t is unknown
- Dürr–Høyer minimum-finding algorithm, $O(\sqrt{N})$
- Analysis: rank probability, expected runtime, success probability
- Extension: find k smallest items in $O(\sqrt{kN})$
- A cleaner reformulation by Miyamoto, Iwamura, Kise (2019)

Takeaway

Grover is not only a search primitive; with the right oracle design it becomes an optimization primitive.

Outline

1. Motivation
2. Quantum Exponential Searching
3. The Minimum Finding Algorithm
4. Analysis
5. Finding k Minima
6. Applications and Wrap-up
7. References

Outline

1. Motivation
2. Quantum Exponential Searching
3. The Minimum Finding Algorithm
4. Analysis
5. Finding k Minima
6. Applications and Wrap-up
7. References

Grover Recap

- Given $f : \{0,1\}^n \to \{0,1\}$ and the promise that at least one x satisfies $f(x) = 1$
- Grover finds such x in $O(\sqrt{N})$ queries, where $N = 2^n$
- Classically we need $\Theta(N)$, and BBBV shows $\sqrt{N}$ is optimal

Now, instead of "marked or not", let's learn an algorithm that finds the min/max!

Finding the Minimum

5	6	4	7	2
$101_{(2)}$	$110_{(2)}$	$100_{(2)}$	$111_{(2)}$	$010_{(2)}$

From Search to Optimization

- Problem: given values $T[0], \ldots, T[N-1]$, find

$$y^* = \arg \min_{0 \le y \le N-1} T[y]$$

- Classically: $\Theta(N)$ comparisons are necessary and sufficient
- Quantum question: can we do it in $O(\sqrt{N})$?

Yes. Dürr and Høyer (1996) gave a surprisingly short algorithm [DH96].

The paper is only **2 pages long**.

Today's Algorithm

A quantum algorithm for finding the minimum*

Christoph Dürr† Peter Høyer‡

July 1996

1 Introduction

Let $T[0..N-1]$ be an unsorted table of N items, each holding a value from an ordered set. For simplicity, assume that all values are distinct. The minimum searching problem is to find the index y such that $T[y]$ is minimum. This clearly requires a linear number of probes on a classical probabilistic Turing machine.

Here, we give a simple quantum algorithm which solves the problem using $\mathcal{O}(\sqrt{N})$ probes. The main subroutine is the quantum exponential searching algorithm of [2], which itself is a generalization of Grover's recent quantum searching algorithm [3]. Due to a general lower bound of [1], this is within a constant factor of the optimum.

2 The algorithm

Our algorithm calls the quantum exponential search-

We first give the minimum searching algorithm, then the proof of the probability of success.

Quantum Minimum Searching Algorithm

1. Choose threshold index $0 \le y \le N-1$ uniformly at random.
2. Repeat the following and interrupt it when the total running time is more than $22.5\sqrt{N} + 1.4\lg^2 N$.[1] Then go to stage 2(2c).
   (a) Initialize the memory as $\sum_j \frac{1}{\sqrt{N}}|j\rangle|y\rangle$. Mark every item j for which $T[j] < T[y]$.
   (b) Apply the quantum exponential searching algorithm of [2].
   (c) Observe the first register: let y' be the outcome. If $T[y'] < T[y]$, then set threshold index y to y'.

This is what we will learn today.

Why Min Finding Matters

- It is one of the simplest non-trivial quantum building blocks
- Appears as a subroutine in many algorithms:
  - k-nearest neighbor
  - clustering, classification
  - shortest path, MST on graphs

It is a simple application of Grover.

Outline

1. Motivation
2. Quantum Exponential Searching
3. The Minimum Finding Algorithm
4. Analysis
5. Finding k Minima
6. Applications and Wrap-up
7. References

Why Use Exponential Searching?

Recall how Grover works.

- To find a marked item with high probability, we apply near $\left\lfloor \frac{\pi}{4}\sqrt{N/t} \right\rfloor$ Grover iterations, where t is the number of marked items.
- Grover assumes we **know t in advance**.

So we would first have to estimate t with quantum counting (runs in $O(\sqrt{N})$, lecture 2) and then feed it into Grover.

Two Ways to Handle Unknown t

There are two ways to run Grover without knowing t.

- **Counting + Grover**: first guess t with quantum counting, then run Grover.
- **Exponential searching**: works without knowing t.

Both take the same time, $O(\sqrt{N/t})$ per call.

Exponential searching is simpler. That is why most people use it as the subroutine.

Grover when t Is Known

target don't care

$$\sqrt{\frac{t}{N}}\,|\bar{1}\rangle + \sqrt{\frac{N-t}{N}}\,|\bar{0}\rangle \qquad \sin^2\theta = \frac{t}{N}$$

same

$$\sin\theta\,|\bar{1}\rangle + \cos\theta\,|\bar{0}\rangle$$

after j grover iteration

$$\sin\left[(2k+1)\theta\right]|\bar{1}\rangle + \cos\left[(2k+1)\theta\right]|\bar{0}\rangle$$

thus, $j \approx \frac{\pi}{4}\sqrt{\frac{N}{k}}$ iteration needed

The Catch: t Unknown

- If we just use $m \approx \frac{\pi}{4}\sqrt{N}$ (good for $t = 1$), the success probability is **not** monotone in j
- For $N = 2^{20}$ and $t = 1$, $m = 804$ iterations succeed almost certainly
- The same 804 iterations on $t = 4$ succeed with probability $< 10^{-6}$

The wrong t drastically affects the success probability.

The Averaging Trick

Idea: don't pick a single j, randomize over a window.

Lemma 1 (BBHT, Lemma 2)

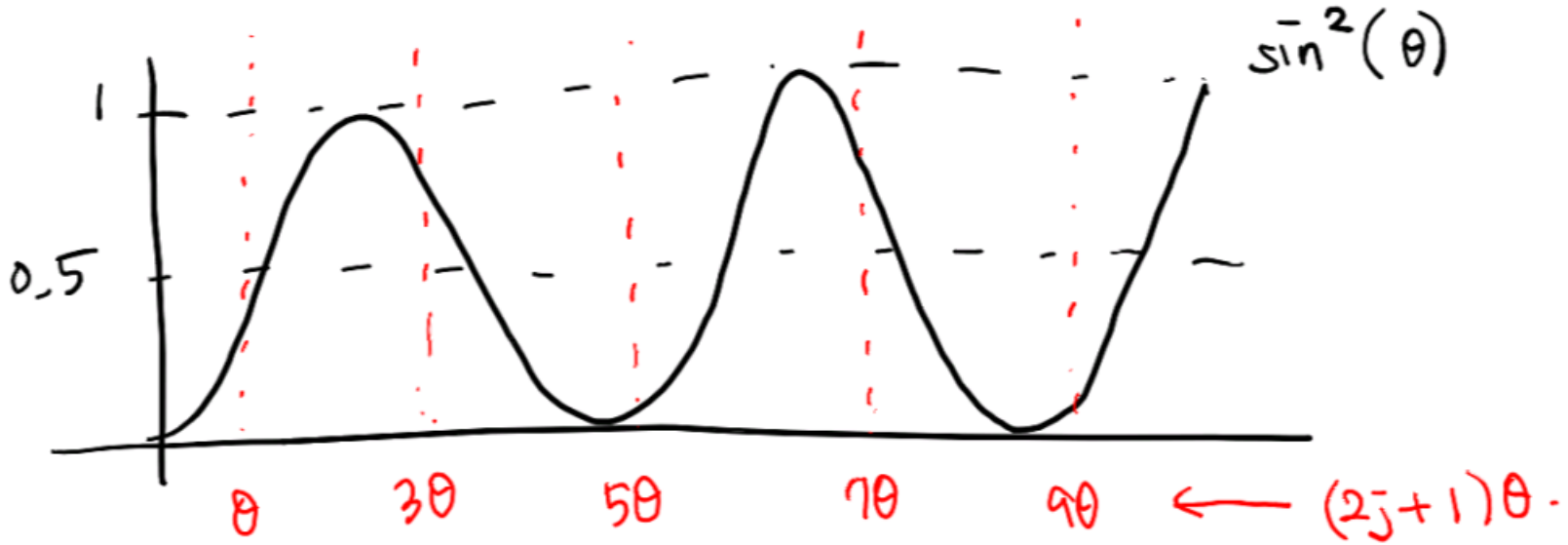
Let $\sin^2\theta = t/N$ *(unknown) and pick* j *uniform in* $\{0, \ldots, m-1\}$. *Apply* j *Grover iterations and measure. Then*

$$P_m = \tfrac{1}{2} - \frac{\sin(4m\theta)}{4m\sin(2\theta)}.$$

In particular, if $m \geq 1/\sin(2\theta)$, then $P_m \geq 1/4$.

Proof: average $\sin^2((2j+1)\theta)$ over j via the cosine-sum identity.

Lemma 1: Visual



If we sample j from $[m]$ (m is sufficiently large)

success prob ~ 0.5.

if $m \geq \frac{1}{\sin 2\theta}$, we get success prob $\geq \frac{1}{4}$

The BBHT Exponential Searching Algorithm

Inputs: oracle f, no knowledge of t. Choose any $\lambda \in (1, 4/3)$.

1. Set $m \leftarrow 1$
2. Pick j uniformly at random in $\{0, \ldots, m-1\}$
3. Apply j Grover iterations to the equal superposition; measure i
4. If $f(i) = 1$, return i
5. Else set $m \leftarrow \min(\lceil \lambda m \rceil, \sqrt{N})$ and go to step 2

The window m grows by a factor of λ until it covers the critical scale $m_0 = 1/\sin(2\theta) \approx \frac{1}{2}\sqrt{N/t}$, where every round succeeds with probability $\geq 1/4$.

BBHT Runtime

Theorem 2 (BBHT, Theorem 3)

For any $1 \leq t \leq 3N/4$, the algorithm returns a marked item in expected

$$O\left(\sqrt{N/t}\right)$$

Grover iterations, even though t is unknown.

Let $m_0 = 1/\sin(2\theta)$. Each round in round s uses $m = \lceil \lambda^{s-1} \rceil$ as its window and costs $\frac{m}{2}$ Grover iterations on average.

- **Phase 1 (ramp-up)**: m grows by factor λ until $m \geq m_0$. Geometric sum gives total cost $\leq \dfrac{1}{2} \cdot \dfrac{\lambda}{\lambda - 1} m_0$
- **Phase 2 (critical stage)**: each round succeeds with prob. $\geq 1/4$. The dominant (last) round has cost $\leq \dfrac{\lambda}{8 - 6\lambda} m_0$

Plugging in λ

Adding the two phases,

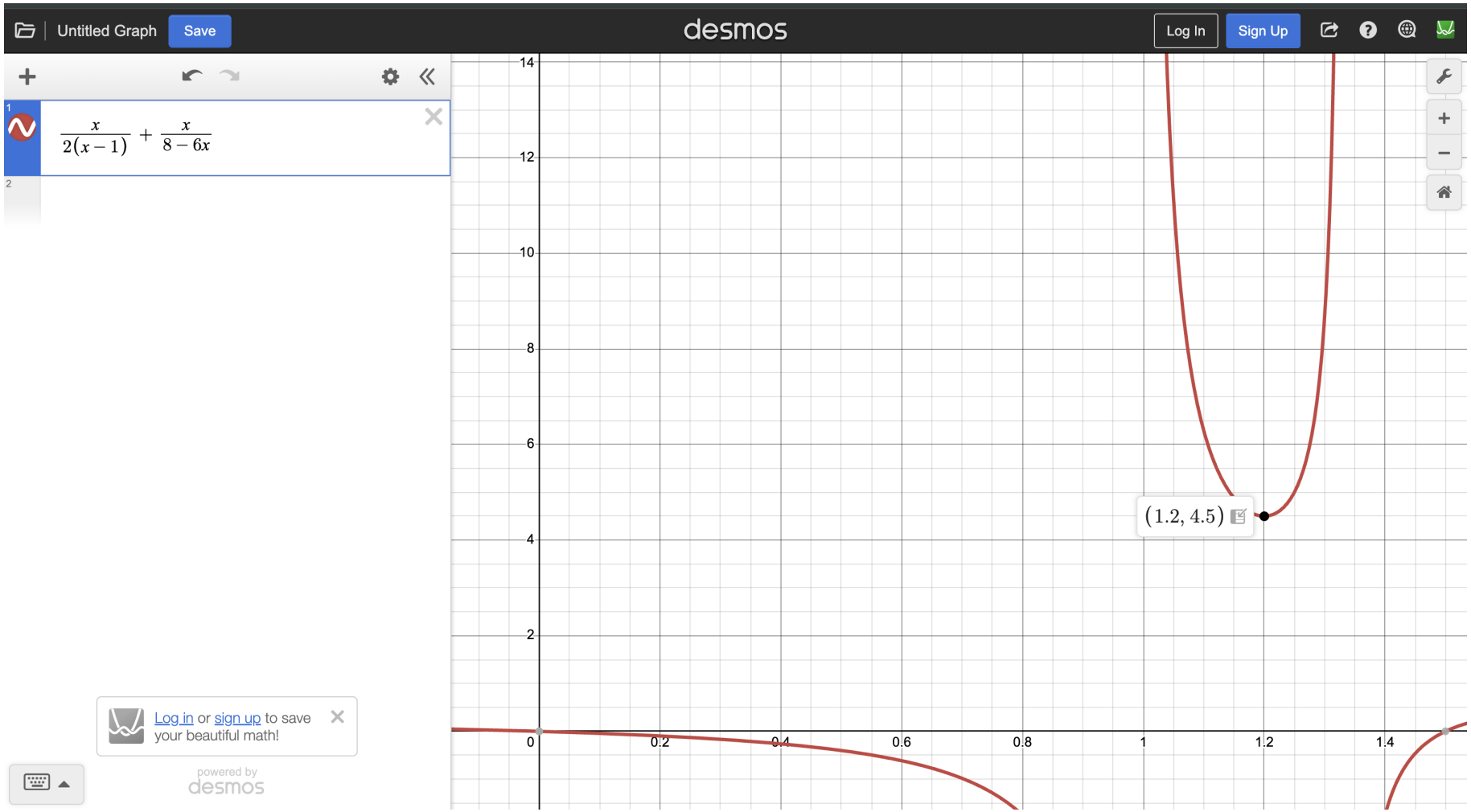
$$\text{(expected iterations)} \;\leq\; C(\lambda)\, m_0, \qquad C(\lambda) \;=\; \frac{1}{2} \cdot \frac{\lambda}{\lambda - 1} \;+\; \frac{\lambda}{8 - 6\lambda}.$$

Now, substitute $C(6/5) = \frac{9}{2}$.

Using $m_0 \leq \sqrt{N/t}$ (valid for $1 \leq t \leq 3N/4$):

The algorithm uses at most $\frac{9}{2} m_0 \leq \frac{9}{2}\sqrt{N/t}$ Grover iterations in expectation, with no prior knowledge of t.

$C(\lambda)$ over $(1, 4/3)$



We need at most $\frac{9}{2}m_0 \leq \frac{9}{2}\sqrt{N/t}$ Grover iterations.

What About $t = 0$?

- If no item is marked, step 4 never fires — the algorithm never terminates by itself.
- Step 5 caps the window at $m = \sqrt{N}$, so the ramp-up gets stuck there. After that, every loop pass simply runs at the maximum window size.
- At $m = \sqrt{N}$, *if* there were even one marked item ($t \geq 1$), each pass would still find it with probability $\geq 1/4$.
- So we wrap the algorithm in a **time-out**: run a constant number r of passes at $m = \sqrt{N}$. If none returns a marked item, declare "$t = 0$".
- The false-negative probability is at most $(3/4)^r$ — exponentially small in r.

This is exactly how Dürr–Høyer detects "y is already the minimum".

Black-Box Interface

For the rest of the lecture, treat exponential searching as a primitive:

ExpSearch(f)

Input: oracle f on $\{0, \ldots, N-1\}$ with $t \geq 1$ marked items.

Output: a uniformly random marked index.

Cost: $\frac{9}{2}\sqrt{N/t}$ expected Grover iterations.

Two facts we will use:

- output is uniform over marked items
- each call is independent; cost depends only on the current t

Outline

1. Motivation
2. Quantum Exponential Searching
3. The Minimum Finding Algorithm
4. Analysis
5. Finding k Minima
6. Applications and Wrap-up
7. References

Problem Setup

Definition 3 (Minimum finding)

Let $T[0..N-1]$ be an unsorted table of N items from an ordered set.

Access: query oracle $|x\rangle|0\rangle \mapsto |x\rangle|T[x]\rangle$.

Goal: find y^* such that $T[y^*] = \min_x T[x]$.

Assume values are distinct for simplicity; the analysis extends to ties.

The Threshold Oracle

Given a current "best-so-far" index y, define

$$f_y(x) = \begin{cases} 1, & T[x] < T[y] \\ 0, & T[x] \geq T[y] \end{cases}$$

- f_y marks exactly the items **strictly better** than y
- Implemented by one call to T plus a comparator: $O_g(\log N)$ extra gates
- Let t = number of marked items. Then t is the current rank of y minus one

The Threshold Oracle

Given a current "best-so-far" index y, define

$$f_y(x) = \begin{cases} 1, & T[x] < T[y] \\ 0, & T[x] \geq T[y] \end{cases}$$

- f_y marks exactly the items **strictly better** than y
- Implemented by one call to T plus a comparator: $O_g(\log N)$ extra gates
- Let t = number of marked items. Then t is the current rank of y minus one

The Threshold Oracle

Given a current "best-so-far" index y, define

$$f_y(x) = \begin{cases} 1, & T[x] < T[y] \\ 0, & T[x] \geq T[y] \end{cases}$$

- f_y marks exactly the items **strictly better** than y
- Implemented by one call to T plus a comparator: $O_g(\log N)$ extra gates
- Let t = number of marked items. Then t is the current rank of y minus one

The Threshold Oracle

Given a current "best-so-far" index y, define

$$f_y(x) = \begin{cases} 1, & T[x] < T[y] \\ 0, & T[x] \geq T[y] \end{cases}$$

- f_y marks exactly the items **strictly better** than y
- Implemented by one call to T plus a comparator: $O_g(\log N)$ extra gates
- Let t = number of marked items. Then t is the current rank of y minus one

Feed f_y into ExpSearch, keep the winner, repeat.

Algorithmic Idea

- Initialize y at random
- Run $\textsc{ExpSearch}(f_y)$ to get some y' with $T[y'] < T[y]$
- Update $y \leftarrow y'$
- The threshold strictly decreases at each successful round
- Repeat until no item is better than y; return y

Algorithmic Idea

- Initialize y at random
- Run $\textsc{ExpSearch}(f_y)$ to get some y' with $T[y'] < T[y]$
- Update $y \leftarrow y'$
- The threshold strictly decreases at each successful round
- Repeat until no item is better than y; return y

Algorithmic Idea

- Initialize y at random
- Run $\textsc{ExpSearch}(f_y)$ to get some y' with $T[y'] < T[y]$
- Update $y \leftarrow y'$
- The threshold strictly decreases at each successful round
- Repeat until no item is better than y; return y

Algorithmic Idea

- Initialize y at random
- Run $\textsc{ExpSearch}(f_y)$ to get some y' with $T[y'] < T[y]$
- Update $y \leftarrow y'$
- The threshold strictly decreases at each successful round
- Repeat until no item is better than y; return y

Algorithmic Idea

- Initialize y at random
- Run $\textsc{ExpSearch}(f_y)$ to get some y' with $T[y'] < T[y]$
- Update $y \leftarrow y'$
- The threshold strictly decreases at each successful round
- Repeat until no item is better than y; return y

Algorithmic Idea

- Initialize y at random
- Run $\textsc{ExpSearch}(f_y)$ to get some y' with $T[y'] < T[y]$
- Update $y \leftarrow y'$
- The threshold strictly decreases at each successful round
- Repeat until no item is better than y; return y

The number of marked items t shrinks over time,
so each round becomes "easier but with fewer targets."

Visualizing the Threshold Descent

A quantum algorithm for finding the minimum*

Christoph Dürr† Peter Høyer‡

July 1996

1 Introduction

Let $T[0..N-1]$ be an unsorted table of N items, each holding a value from an ordered set. For simplicity, assume that all values are distinct. The minimum searching problem is to find the index y such that $T[y]$ is minimum. This clearly requires a linear number of probes on a classical probabilistic Turing machine.

Here, we give a simple quantum algorithm which solves the problem using $\mathcal{O}(\sqrt{N})$ probes. The main subroutine is the quantum exponential searching algorithm of [2], which itself is a generalization of Grover's recent quantum searching algorithm [3]. Due to a general lower bound of [1], this is within a constant factor of the optimum.

2 The algorithm

Our algorithm calls the quantum exponential search-

We first give the minimum searching algorithm, then the proof of the probability of success.

Quantum Minimum Searching Algorithm

1. Choose threshold index $0 \le y \le N-1$ uniformly at random.
2. Repeat the following and interrupt it when the total running time is more than $22.5\sqrt{N} + 1.4\lg^2 N$.[1] Then go to stage 2(2c).
   (a) Initialize the memory as $\sum_j \frac{1}{\sqrt{N}}|j\rangle|y\rangle$. Mark every item j for which $T[j] < T[y]$.
   (b) Apply the quantum exponential searching algorithm of [2].
   (c) Observe the first register: let y' be the outcome. If $T[y'] < T[y]$, then set threshold index y to y'.

The threshold y jumps down to a random marked item at each step, until the set of marked items is empty.

The Dürr–Høyer Algorithm

Output: index y with $T[y]$ minimal, w.h.p.

1. Pick $y \in \{0, \ldots, N-1\}$ uniformly at random
2. Repeat until total time exceeds

$$22.5\sqrt{N} + 1.4 \lg^2 N$$

   1. Prepare $\frac{1}{\sqrt{N}} \sum_j |j\rangle|y\rangle$; mark j iff $T[j] < T[y]$
   2. $y' \leftarrow \textsc{ExpSearch}(f_y)$
   3. If $T[y'] < T[y]$, set $y \leftarrow y'$
3. Return y

Time budget: one Grover iteration = one time step; preparing stage 2(a) costs $\lg N$.

Outline

1. Motivation
2. Quantum Exponential Searching
3. The Minimum Finding Algorithm
4. Analysis
5. Finding k Minima
6. Applications and Wrap-up
7. References

Analysis Roadmap

- Pretend we never time out ("infinite algorithm")
- Show: item of rank r becomes threshold with probability $1/r$
- Sum contributions $\Rightarrow$ expected runtime is $O(\sqrt{N})$
- Markov $\Rightarrow$ running $2\times$ longer gives success probability $\geq 1/2$

Analysis Roadmap

- Pretend we never time out ("infinite algorithm")
- Show: item of rank r becomes threshold with probability $1/r$
- Sum contributions $\Rightarrow$ expected runtime is $O(\sqrt{N})$
- Markov $\Rightarrow$ running $2\times$ longer gives success probability $\geq 1/2$

Analysis Roadmap

- Pretend we never time out ("infinite algorithm")
- Show: item of rank r becomes threshold with probability $1/r$
- Sum contributions $\Rightarrow$ expected runtime is $O(\sqrt{N})$
- Markov $\Rightarrow$ running $2\times$ longer gives success probability $\geq 1/2$

Analysis Roadmap

- Pretend we never time out ("infinite algorithm")
- Show: item of rank r becomes threshold with probability $1/r$
- Sum contributions $\Rightarrow$ expected runtime is $O(\sqrt{N})$
- Markov $\Rightarrow$ running $2\times$ longer gives success probability $\geq 1/2$

Analysis Roadmap

- Pretend we never time out ("infinite algorithm")
- Show: item of rank r becomes threshold with probability $1/r$
- Sum contributions $\Rightarrow$ expected runtime is $O(\sqrt{N})$
- Markov $\Rightarrow$ running $2\times$ longer gives success probability $\geq 1/2$

Everything follows from one clean lemma on rank probabilities.

Lemma 1: Rank Probability

Lemma 4 (Dürr–Høyer)

Let $p(t, r)$ be the probability that the item of rank r ever becomes the threshold, when the infinite algorithm starts with t marked items. Then

$$p(t, r) = \begin{cases} 1/r, & r \leq t \\ 0, & r > t \end{cases}$$

- Rank r: the r-th smallest element
- Remarkably, the probability does not depend on t
- Rank 1 (the true minimum) is hit with probability 1

Proof of Lemma 1

Induction on t for fixed r.

Base: $p(r, r) = 1/r$ because AA returns a uniform marked index.

Step: assume $p(k, r) = 1/r$ for all $k \in [r, t]$. With $t + 1$ marked items:

$$p(t+1, r) = \underbrace{\tfrac{1}{t+1}}_{\text{hit } r \text{ now}} + \sum_{k=r+1}^{t+1} \tfrac{1}{t+1}\, p(k-1, r)$$

$$= \tfrac{1}{t+1} + \tfrac{t+1-r}{t+1} \cdot \tfrac{1}{r} = \tfrac{1}{r}.$$

Cases where the chosen item has rank $< r$ contribute 0 (rank r stops being marked).

Lemma 2: Expected Runtime

Lemma 5

The expected total time used by the infinite algorithm before y holds the index of the minimum is at most

$$m_0 = \frac{45}{4}\sqrt{N} + \frac{7}{10}\lg^2 N.$$

Two pieces:

- **AA calls:** $\sum_r p(N, r) \cdot \frac{9}{2}\sqrt{N/(r-1)}$
- **Oracle prep:** $\sum_r p(N, r) \cdot \lg N$

Proof of Lemma 2 (AA part)

$$\sum_{r=2}^{N} p(N,r)\,\tfrac{9}{2}\sqrt{\tfrac{N}{r-1}} = \tfrac{9}{2}\sqrt{N}\sum_{r=1}^{N-1}\tfrac{1}{r+1}\cdot\tfrac{1}{\sqrt{r}}$$

$$\leq \tfrac{9}{2}\sqrt{N}\left(\tfrac{1}{2}+\sum_{r=2}^{N-1} r^{-3/2}\right) \leq \tfrac{9}{2}\sqrt{N}\left(\tfrac{1}{2}+\int_{1}^{N-1} r^{-3/2}\,dr\right)$$

$$= \tfrac{9}{2}\sqrt{N}\left(\tfrac{1}{2}+\left[-2r^{-1/2}\right]_{1}^{N-1}\right) \leq \tfrac{45}{4}\sqrt{N}.$$

For oracle prep: $\sum_r p(N,r)\lg N = (H_N - 1)\lg N \leq \ln N \lg N \leq \frac{7}{10}\lg^2 N$.

Theorem 1: Success Probability

Theorem 6 (Dürr–Høyer [DH96])

With time-out $2m_0 = 22.5\sqrt{N} + 1.4\lg^2 N$*, the algorithm outputs the index of the minimum with probability at least* $1/2$*.*

Its running time is $O(\sqrt{N})$*.*

- Markov: $\Pr[\text{runtime} > 2m_0] \leq 1/2$
- So with probability $\geq 1/2$ the infinite algorithm has already reached the minimum
- Boosting: run c copies, take the best $\Rightarrow$ error $\leq 1/2^c$

Theorem 1: Success Probability

Theorem 6 (Dürr–Høyer [DH96])

With time-out $2m_0 = 22.5\sqrt{N} + 1.4\lg^2 N$*, the algorithm outputs the index of the minimum with probability at least* $1/2$*.*

Its running time is $O(\sqrt{N})$*.*

- Markov: $\Pr[\text{runtime} > 2m_0] \leq 1/2$
- So with probability $\geq 1/2$ the infinite algorithm has already reached the minimum
- Boosting: run c copies, take the best $\Rightarrow$ error $\leq 1/2^c$

Theorem 1: Success Probability

Theorem 6 (Dürr–Høyer [DH96])

With time-out $2m_0 = 22.5\sqrt{N} + 1.4\lg^2 N$*, the algorithm outputs the index of the minimum with probability at least* $1/2$*.*

Its running time is $O(\sqrt{N})$*.*

- Markov: $\Pr[\text{runtime} > 2m_0] \leq 1/2$
- So with probability $\geq 1/2$ the infinite algorithm has already reached the minimum
- Boosting: run c copies, take the best $\Rightarrow$ error $\leq 1/2^c$

Why This Is Tight

- Min finding $\geq$ unstructured search (a marked item is just the minimum of a 0/1 table)
- By BBBV [BBBV97], $\Omega(\sqrt{N})$ queries are required
- So Dürr–Høyer is **asymptotically optimal**

Why This Is Tight

- Min finding $\geq$ unstructured search (a marked item is just the minimum of a 0/1 table)
- By BBBV [BBBV97], $\Omega(\sqrt{N})$ queries are required
- So Dürr–Høyer is **asymptotically optimal**

Why This Is Tight

- Min finding $\geq$ unstructured search (a marked item is just the minimum of a 0/1 table)
- By BBBV [BBBV97], $\Omega(\sqrt{N})$ queries are required
- So Dürr–Høyer is **asymptotically optimal**

Why This Is Tight

- Min finding $\geq$ unstructured search (a marked item is just the minimum of a 0/1 table)
- By BBBV [BBBV97], $\Omega(\sqrt{N})$ queries are required
- So Dürr–Høyer is **asymptotically optimal**

Grover is the lower bound; Dürr–Høyer matches it for a strictly harder task.

What About the Maximum?

- Flip the comparator: $f_y(x) = 1$ iff $T[x] > T[y]$
- Same algorithm, same analysis, same $O(\sqrt{N})$
- Ahuja–Kapoor gave an explicit write-up [AK99]

What About the Maximum?

- Flip the comparator: $f_y(x) = 1$ iff $T[x] > T[y]$
- Same algorithm, same analysis, same $O(\sqrt{N})$
- Ahuja–Kapoor gave an explicit write-up [AK99]

What About the Maximum?

- Flip the comparator: $f_y(x) = 1$ iff $T[x] > T[y]$
- Same algorithm, same analysis, same $O(\sqrt{N})$
- Ahuja–Kapoor gave an explicit write-up [AK99]

What About the Maximum?

- Flip the comparator: $f_y(x) = 1$ iff $T[x] > T[y]$
- Same algorithm, same analysis, same $O(\sqrt{N})$
- Ahuja–Kapoor gave an explicit write-up [AK99]

So we have **min, max, arg-min, arg-max** all in $O(\sqrt{N})$.

Quick check

Any issues with Lemma 1 or the runtime bound before we go to k-minima?

Outline

1. Motivation
2. Quantum Exponential Searching
3. The Minimum Finding Algorithm
4. Analysis
5. Finding k Minima
6. Applications and Wrap-up
7. References

The k-Minima Problem

Definition 7 (k-minima)

Given $T[0..N-1]$ and $k \leq N$, return the indices of the k smallest values.

- Naive quantum: run Dürr–Høyer k times $\Rightarrow O(k\sqrt{N})$
- Classical lower bound: $\Omega(N)$ (must read everything)
- Can we do **better than** $k\sqrt{N}$?

The k-Minima Problem

Definition 7 (k-minima)

Given $T[0..N-1]$ and $k \leq N$, return the indices of the k smallest values.

- Naive quantum: run Dürr–Høyer k times $\Rightarrow O(k\sqrt{N})$
- Classical lower bound: $\Omega(N)$ (must read everything)
- Can we do **better than** $k\sqrt{N}$?

The k-Minima Problem

Definition 7 (k-minima)

Given $T[0..N-1]$ and $k \leq N$, return the indices of the k smallest values.

- Naive quantum: run Dürr–Høyer k times $\Rightarrow O(k\sqrt{N})$
- Classical lower bound: $\Omega(N)$ (must read everything)
- Can we do **better than** $k\sqrt{N}$?

The k-Minima Problem

Definition 7 (k-minima)

Given $T[0..N-1]$ and $k \leq N$, return the indices of the k smallest values.

- Naive quantum: run Dürr–Høyer k times $\Rightarrow O(k\sqrt{N})$
- Classical lower bound: $\Omega(N)$ (must read everything)
- Can we do **better than** $k\sqrt{N}$?

Yes: $O(\sqrt{kN})$ is achievable and optimal.

Why $\sqrt{kN}$ Is the Right Scale

Think of finding marked items one by one:

$$\underbrace{\sqrt{N/k}}_{\text{1st}} + \underbrace{\sqrt{N/(k-1)}}_{\text{2nd}} + \cdots + \underbrace{\sqrt{N}}_{k\text{-th}}$$

Bound the sum by an integral:

$$\sum_{t=1}^{k} \sqrt{\tfrac{N}{t}} \leq \sqrt{N}\left(1 + \int_1^k t^{-1/2}\, dt\right) = O(\sqrt{kN}).$$

Why $\sqrt{kN}$ Is the Right Scale

Think of finding marked items one by one:

$$\underbrace{\sqrt{N/k}}_{\text{1st}} + \underbrace{\sqrt{N/(k-1)}}_{\text{2nd}} + \cdots + \underbrace{\sqrt{N}}_{k\text{-th}}$$

Bound the sum by an integral:

$$\sum_{t=1}^{k} \sqrt{\tfrac{N}{t}} \leq \sqrt{N}\left(1 + \int_1^k t^{-1/2}\,dt\right) = O(\sqrt{kN}).$$

Each find gets cheaper as k shrinks; the sum telescopes into $\sqrt{kN}$.

Dürr–Heiligman Approach (2006)

Maintain a set T of k thresholds and an oracle

$$f_T(x) = \begin{cases} 1, & T[x] < T[t] \text{ for some } t \in T \\ 0, & \text{otherwise} \end{cases}$$

- Initialize T with k random indices
- Loop: find x with $f_T(x) = 1$; replace $\arg\max_{t \in T} T[t]$ with x
- Greedy swap on the *worst* current threshold [DHHM06]
- Keep T duplicate-free by removing it from the marked set

Dürr–Heiligman Approach (2006)

Maintain a set T of k thresholds and an oracle

$$f_T(x) = \begin{cases} 1, & T[x] < T[t] \text{ for some } t \in T \\ 0, & \text{otherwise} \end{cases}$$

- Initialize T with k random indices
- Loop: find x with $f_T(x) = 1$; replace $\arg\max_{t \in T} T[t]$ with x
- Greedy swap on the *worst* current threshold [DHHM06]
- Keep T duplicate-free by removing it from the marked set

Dürr–Heiligman Approach (2006)

Maintain a set T of k thresholds and an oracle

$$f_T(x) = \begin{cases} 1, & T[x] < T[t] \text{ for some } t \in T \\ 0, & \text{otherwise} \end{cases}$$

- Initialize T with k random indices
- Loop: find x with $f_T(x) = 1$; replace $\arg\max_{t \in T} T[t]$ with x
- Greedy swap on the *worst* current threshold [DHHM06]
- Keep T duplicate-free by removing it from the marked set

Dürr–Heiligman Approach (2006)

Maintain a set T of k thresholds and an oracle

$$f_T(x) = \begin{cases} 1, & T[x] < T[t] \text{ for some } t \in T \\ 0, & \text{otherwise} \end{cases}$$

- Initialize T with k random indices
- Loop: find x with $f_T(x) = 1$; replace $\arg\max_{t \in T} T[t]$ with x
- Greedy swap on the *worst* current threshold [DHHM06]
- Keep T duplicate-free by removing it from the marked set

Dürr–Heiligman Approach (2006)

Maintain a set T of k thresholds and an oracle

$$f_T(x) = \begin{cases} 1, & T[x] < T[t] \text{ for some } t \in T \\ 0, & \text{otherwise} \end{cases}$$

- Initialize T with k random indices
- Loop: find x with $f_T(x) = 1$; replace $\arg\max_{t \in T} T[t]$ with x
- Greedy swap on the *worst* current threshold [DHHM06]
- Keep T duplicate-free by removing it from the marked set

Correct analysis is subtle because the oracle is entangled with the whole set T.

A Simpler Reformulation

Miyamoto–Iwamura–Kise (2019) split the task into two clean phases [MIK19].

- **Phase 1:** find a *single* threshold $t_{k'}$ such that

$$\left|\{x : T[x] < T[t_{k'}]\}\right| = k' \geq k$$

- **Phase 2:** run "find all marked k' indices" using that single threshold

A Simpler Reformulation

Miyamoto–Iwamura–Kise (2019) split the task into two clean phases [MIK19].

- **Phase 1:** find a *single* threshold $t_{k'}$ such that

$$\left|\{x : T[x] < T[t_{k'}]\}\right| = k' \geq k$$

- **Phase 2:** run "find all marked k' indices" using that single threshold

A Simpler Reformulation

Miyamoto–Iwamura–Kise (2019) split the task into two clean phases [MIK19].

- **Phase 1:** find a *single* threshold $t_{k'}$ such that

$$\left|\{x : T[x] < T[t_{k'}]\}\right| = k' \geq k$$

- **Phase 2:** run "find all marked k' indices" using that single threshold

No k-threshold oracle, no greedy bookkeeping. Just FM + quantum counting + AA.

Phase 1: Finding the Good Threshold

- Run Dürr–Høyer and **remember** the last k threshold indices $t_1^{\mathrm{FM}}, \ldots$
- Counting oracle $h(t) = |\{x : T[x] < T[t]\}|$ by quantum counting, $O(\sqrt{N})$ per query [BHT98]
- Binary search on k candidate thresholds: find i with

$$h(t_{i+1}^{\mathrm{FM}}) \leq k < h(t_i^{\mathrm{FM}})$$

- Cost: $O(\sqrt{N} \log k)$

Phase 1: Finding the Good Threshold

- Run Dürr–Høyer and **remember** the last k threshold indices $t_1^{\mathrm{FM}}, \ldots$
- Counting oracle $h(t) = |\{x : T[x] < T[t]\}|$ by quantum counting, $O(\sqrt{N})$ per query [BHT98]
- Binary search on k candidate thresholds: find i with

$$h(t_{i+1}^{\mathrm{FM}}) \leq k < h(t_i^{\mathrm{FM}})$$

- Cost: $O(\sqrt{N} \log k)$

Phase 1: Finding the Good Threshold

- Run Dürr–Høyer and **remember** the last k threshold indices $t_1^{\mathrm{FM}}, \ldots$
- Counting oracle $h(t) = |\{x : T[x] < T[t]\}|$ by quantum counting, $O(\sqrt{N})$ per query [BHT98]
- Binary search on k candidate thresholds: find i with

$$h(t_{i+1}^{\mathrm{FM}}) \leq k < h(t_i^{\mathrm{FM}})$$

- Cost: $O(\sqrt{N} \log k)$

Phase 1: Finding the Good Threshold

- Run Dürr–Høyer and **remember** the last k threshold indices $t_1^{\mathrm{FM}}, \ldots$
- Counting oracle $h(t) = |\{x : T[x] < T[t]\}|$ by quantum counting, $O(\sqrt{N})$ per query [BHT98]
- Binary search on k candidate thresholds: find i with

$$h(t_{i+1}^{\mathrm{FM}}) \leq k < h(t_i^{\mathrm{FM}})$$

- Cost: $O(\sqrt{N} \log k)$

Phase 1: Finding the Good Threshold

- Run Dürr–Høyer and **remember** the last k threshold indices $t_1^{\mathrm{FM}}, \ldots$
- Counting oracle $h(t) = |\{x : T[x] < T[t]\}|$ by quantum counting, $O(\sqrt{N})$ per query [BHT98]
- Binary search on k candidate thresholds: find i with

$$h(t_{i+1}^{\mathrm{FM}}) \leq k < h(t_i^{\mathrm{FM}})$$

- Cost: $O(\sqrt{N} \log k)$

FM gives us a geometric sequence of thresholds for free; we only have to count.

Phase 2: Find All Marked Indices

Fixed threshold $t_{k'}$. Oracle marks the k' smallest items:

$$f'_{t_{k'}}(x) = \begin{cases} 1, & T[x] < T[t_{k'}] \text{ and } x \notin T_{\text{found}} \\ 0, & \text{otherwise} \end{cases}$$

- After finding t of them, $(k' - t)$ remain marked out of N
- t-th call to AA: $O(\sqrt{N/(k' - t)})$
- Total: $O(\sqrt{k'N}) = O(\sqrt{kN})$

Phase 2: Find All Marked Indices

Fixed threshold $t_{k'}$. Oracle marks the k' smallest items:

$$f'_{t_{k'}}(x) = \begin{cases} 1, & T[x] < T[t_{k'}] \text{ and } x \notin T_{\text{found}} \\ 0, & \text{otherwise} \end{cases}$$

- After finding t of them, $(k' - t)$ remain marked out of N
- t-th call to AA: $O(\sqrt{N/(k' - t)})$
- Total: $O(\sqrt{k'N}) = O(\sqrt{kN})$

Phase 2: Find All Marked Indices

Fixed threshold $t_{k'}$. Oracle marks the k' smallest items:

$$f'_{t_{k'}}(x) = \begin{cases} 1, & T[x] < T[t_{k'}] \text{ and } x \notin T_{\text{found}} \\ 0, & \text{otherwise} \end{cases}$$

- After finding t of them, $(k' - t)$ remain marked out of N
- t-th call to AA: $O(\sqrt{N/(k' - t)})$
- Total: $O(\sqrt{k'N}) = O(\sqrt{kN})$

Putting It Together

Theorem 8 (Miyamoto–Iwamura–Kise [MIK19])

k-minima can be solved with quantum query complexity

$$O(\sqrt{N}\log k) + O(\sqrt{kN}) = O(\sqrt{kN}).$$

- Same asymptotic as Dürr–Heiligman
- But the algorithm is modular: FM $\to$ QC $\to$ AA
- Each subroutine is independently analyzed; no joint oracle over k thresholds

Putting It Together

Theorem 8 (Miyamoto–Iwamura–Kise [MIK19])

k-minima can be solved with quantum query complexity

$$O(\sqrt{N}\log k) + O(\sqrt{kN}) = O(\sqrt{kN}).$$

- Same asymptotic as Dürr–Heiligman
- But the algorithm is modular: FM $\to$ QC $\to$ AA
- Each subroutine is independently analyzed; no joint oracle over k thresholds

Putting It Together

Theorem 8 (Miyamoto–Iwamura–Kise [MIK19])

k-minima can be solved with quantum query complexity

$$O(\sqrt{N}\log k) + O(\sqrt{kN}) = O(\sqrt{kN}).$$

- Same asymptotic as Dürr–Heiligman
- But the algorithm is modular: FM $\to$ QC $\to$ AA
- Each subroutine is independently analyzed; no joint oracle over k thresholds

Outline

1. Motivation
2. Quantum Exponential Searching
3. The Minimum Finding Algorithm
4. Analysis
5. Finding k Minima
6. Applications and Wrap-up
7. References

Where Min / k-Min Show Up

- **k-nearest neighbor:** find k closest points to a query under a distance oracle
- **Clustering:** nearest-centroid assignments, Lloyd-style updates
- **Graph problems:** minimum-weight edge, single-source shortest path
- **Optimization inner loops:** any "pick the best candidate" step

Where Min / k-Min Show Up

- **k-nearest neighbor:** find k closest points to a query under a distance oracle
- **Clustering:** nearest-centroid assignments, Lloyd-style updates
- **Graph problems:** minimum-weight edge, single-source shortest path
- **Optimization inner loops:** any "pick the best candidate" step

Where Min / k-Min Show Up

- **k-nearest neighbor:** find k closest points to a query under a distance oracle
- **Clustering:** nearest-centroid assignments, Lloyd-style updates
- **Graph problems:** minimum-weight edge, single-source shortest path
- **Optimization inner loops:** any "pick the best candidate" step

Where Min / k-Min Show Up

- k-**nearest neighbor:** find k closest points to a query under a distance oracle
- **Clustering:** nearest-centroid assignments, Lloyd-style updates
- **Graph problems:** minimum-weight edge, single-source shortest path
- **Optimization inner loops:** any "pick the best candidate" step

Where Min / k-Min Show Up

- **k-nearest neighbor:** find k closest points to a query under a distance oracle
- **Clustering:** nearest-centroid assignments, Lloyd-style updates
- **Graph problems:** minimum-weight edge, single-source shortest path
- **Optimization inner loops:** any "pick the best candidate" step

Min finding is a quadratic speedup you can drop into almost any loop.

What to Take Away

- Min finding = Grover with a shrinking marked set
- Key design trick: the **threshold oracle**, not a new amplitude technique
- Clean analysis hinges on a single identity: rank r is hit with probability $1/r$
- k-min extends the same idea, and becomes modular once we split threshold-finding from enumeration
- All of this is **optimal** up to constants

What to Take Away

- Min finding = Grover with a shrinking marked set
- Key design trick: the **threshold oracle**, not a new amplitude technique
- Clean analysis hinges on a single identity: rank r is hit with probability $1/r$
- k-min extends the same idea, and becomes modular once we split threshold-finding from enumeration
- All of this is **optimal** up to constants

What to Take Away

- Min finding = Grover with a shrinking marked set
- Key design trick: the **threshold oracle**, not a new amplitude technique
- Clean analysis hinges on a single identity: rank r is hit with probability $1/r$
- k-min extends the same idea, and becomes modular once we split threshold-finding from enumeration
- All of this is **optimal** up to constants

What to Take Away

- Min finding = Grover with a shrinking marked set
- Key design trick: the **threshold oracle**, not a new amplitude technique
- Clean analysis hinges on a single identity: rank r is hit with probability $1/r$
- k-min extends the same idea, and becomes modular once we split threshold-finding from enumeration
- All of this is **optimal** up to constants

What to Take Away

- Min finding = Grover with a shrinking marked set
- Key design trick: the **threshold oracle**, not a new amplitude technique
- Clean analysis hinges on a single identity: rank r is hit with probability $1/r$
- k-min extends the same idea, and becomes modular once we split threshold-finding from enumeration
- All of this is **optimal** up to constants

What to Take Away

- Min finding = Grover with a shrinking marked set
- Key design trick: the **threshold oracle**, not a new amplitude technique
- Clean analysis hinges on a single identity: rank r is hit with probability $1/r$
- k-min extends the same idea, and becomes modular once we split threshold-finding from enumeration
- All of this is **optimal** up to constants

Next week: quantum analog-digital conversion and the swap test.

Quiz time!

Outline

1. Motivation
2. Quantum Exponential Searching
3. The Minimum Finding Algorithm
4. Analysis
5. Finding k Minima
6. Applications and Wrap-up
7. References

References I

[AK99] Ashish Ahuja and Sanjiv Kapoor.
A quantum algorithm for finding the maximum, 1999.

[BBBV97] Charles H. Bennett, Ethan Bernstein, Gilles Brassard, and Umesh V. Vazirani.
Strengths and weaknesses of quantum computing.
SIAM Journal on Computing, 26(5):1510–1523, 1997.

[BBHT98] Michel Boyer, Gilles Brassard, Peter Høyer, and Alain Tapp.
Tight bounds on quantum searching.
Fortschritte der Physik, 46(4-5):493–505, 1998.

[BHMT02] Gilles Brassard, Peter Høyer, Michele Mosca, and Alain Tapp.
Quantum amplitude amplification and estimation.
In *Quantum Computation and Quantum Information*, volume 305, pages 53–74. American Mathematical Society, 2002.

References II

[BHT98] Gilles Brassard, Peter Høyer, and Alain Tapp.
Quantum counting.
In *International Colloquium on Automata, Languages, and Programming*, pages 820–831. Springer, 1998.

[DH96] Christoph Dürr and Peter Høyer.
A quantum algorithm for finding the minimum, 1996.

[DHHM06] Christoph Dürr, Mark Heiligman, Peter Høyer, and Mehdi Mhalla.
Quantum query complexity of some graph problems.
SIAM Journal on Computing, 35(6):1310–1328, 2006.

[MIK19] Kohei Miyamoto, Masakazu Iwamura, and Koichi Kise.
A quantum algorithm for finding k-minima, 2019.